

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет Программной Инженерии и Компьютерной Техники
Специальность: Системное и Прикладное Программное Обеспечение
Дисциплина: Системы Искусственного Интеллекта

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
“Создание базы знаний и выполнение запросов в Prolog”
“Создание онтологии в Protege”
Игра “Ticket To Ride”

Студент: Зубулина Юлия Максимовна (465987)
Группа: Р3312
Преподаватель: Болдырева Елена Александровна (157150)

Содержание:

Введение.....	3
Цель работы:.....	3
1. Анализ предметной области и требований.....	3
1.1 Анализ требований:.....	3
1.2 Требования к базе знаний и онтологии для представления знаний:...	4
1.3 Описание предметной области.....	4
2. Часть 1. Разработка базы знаний на языке Prolog.....	6
2.1 Создание базы знаний:.....	6
2.2 Реализация правил:.....	8
2.3 Реализация запросов.....	11
3. Оценка и интерпретация результатов.....	13
3.1 Результаты выполнения запросов:.....	13
3.2 Анализ результатов выполнения запросов.....	15
4. Часть 2. Создание онтологии в Protege.....	16
4.1 Создание классов онтологии.....	16
4.2 Создание свойств онтологии.....	16
4.3 Создание экземпляров классов.....	18
4.4 Заполнение отношений и значений.....	21
4.5 Визуализация и проверка онтологии.....	23
Заключение.....	27

Введение

Цель работы:

Разработка базы знаний на языке Prolog и изучение принципов представления знаний с использованием фактов, отношений и правил. В качестве предметной области выбраны правила и элементы настольной игры Ticket to Ride. Данная предметная область позволяет представить города, маршруты, игроков, игровые карты и другие элементы игры, а также описать отношения между ними и условия выполнения отдельных игровых действий

В рамках работы необходимо создать базу знаний, содержащую одноаргументные и двухаргументные факты, а также правила логического вывода. На основе созданной базы знаний должны быть выполнены запросы различной сложности, включая запросы с переменными и логическими операторами. Полученные результаты позволят продемонстрировать возможности Prolog для представления знаний и выполнения логического вывода

1. Анализ предметной области и требований

1.1 Анализ требований:

Основными требованиями к системе являются:

1. Возможность представления основных объектов игры: городов, маршрутов, игроков, игровых карт, цветов, очков, перегонов и длины маршрутов.
2. Возможность представления отношений между объектами, например принадлежности игровой карты игроку, соединения городов маршрутом, цвета и длины маршрута.
3. Возможность формулирования логических правил для получения новых сведений на основе имеющихся фактов.
4. Возможность выполнения простых и составных запросов к базе знаний.
5. Использование переменных для поиска объектов, удовлетворяющих заданным условиям.
6. Возможность использования логических операторов, включая логическое “и”, “или” отрицание.
7. Возможность проверки результатов логического вывода и анализа полученных решений.

1.2 Требования к базе знаний и онтологии для представления знаний:

1. База знаний должна обеспечивать структурированное представление информации об игровой предметной области, должна содержать не менее 20 фактов с одним аргументом, факты с двумя аргументами и от 5 до 7 правил.
2. Одноаргументные факты используются для описания объектов и их принадлежности к определенным категориям
3. Двухаргументные факты используются для представления отношений между объектами
4. Правила должны обеспечивать возможность получения новых знаний на основе существующих фактов и отношений
5. Для базы знаний также необходимо предусмотреть запросы различной сложности: простые запросы для проверки отдельных фактов, запросы с переменными, запросы с логическими операторами и запросы, требующие применения правил логического вывода.

1.3 Описание предметной области

В качестве предметной области для разработки базы знаний выбрана настольная игра Ticket to Ride. В рамках работы рассматривается упрощенная модель игровой карты, содержащая города, перегоны между городами, составные маршруты и игроков.

Город в базе знаний представлен как отдельный объект игровой карты. Перегон представляет собой участок между двумя городами и характеризуется цветом и длиной. Составной маршрут включает несколько перегонов и имеет определенное количество бонусных очков. Для игроков также задаются их цвета игровых фишек и выбранные маршруты. Каждый построенный перегон приносит игроку кол-во очков, соответствующее его длине. За выполнение выбранного составного маршрута начисляются дополнительные очки, а за невыполненный маршрут из итогового счета вычитается соответствующее количество очков

Кроме описания объектов, база знаний содержит информацию о занятых перегонах и количестве карт определённых цветов у игроков. На основе этих данных сформулированы правила, позволяющие определить свободные перегоны, проверить выполнение маршрута, рассчитать полученные очки, определить возможность строительства перегона или станции, а также проверить возможность использования перегона.

Таким образом, созданная база знаний содержит как исходные факты, так и правила логического вывода, позволяющие получать новые сведения на основе имеющейся информации.

2. Часть 1. Разработка базы знаний на языке Prolog

2.1 Создание базы знаний:

1. Города

```
city(petrograd).  
city(moscow).  
city(warszawa).  
city(riga).  
city(smolensk).  
city(vilna).
```

2. Игроки

```
player(yuliya).  
player(dasha).  
  
player_route(yuliya, petrograd_warszawa).  
player_route(dasha, moscow_riga).
```

3. Цвет фишек игроков

```
player_color(yuliya, black).  
player_color(dasha, purple).
```

4. Перегоны

```
segment(petrograd_moscow).  
segment(petrograd_vilna).  
segment(moscow_smolensk).  
segment(smolensk_vilna).  
segment(warszawa_vilna).  
segment(riga_vilna).  
  
segment_connects(petrograd_moscow, petrograd, moscow).  
segment_connects(petrograd_vilna, petrograd, vilna).  
segment_connects(moscow_smolensk, moscow, smolensk).  
segment_connects(smolensk_vilna, smolensk, vilna).  
segment_connects(warszawa_vilna, warszawa, vilna).  
segment_connects(riga_vilna, riga, vilna).
```

5. Цвета перегонов

```
color(red).  
color(blue).  
color(green).  
color(yellow).  
color(orange).  
color(white).  
  
segment_color(petrograd_moscow, white).  
segment_color(petrograd_vilna, blue).  
segment_color(moscow_smolensk, orange).  
segment_color(smolensk_vilna, yellow).  
segment_color(warszawa_vilna, red).  
segment_color(riga_vilna, green).
```

6. Длина перегонов

```
segment_length(petrograd_moscow, 4).  
segment_length(petrograd_vilna, 4).  
segment_length(moscow_smolensk, 2).  
segment_length(smolensk_vilna, 3).  
segment_length(warszawa_vilna, 3).  
segment_length(riga_vilna, 4).
```

7. Маршруты

```
route(petrograd_warszawa).
route(moscow_riga).

route_points(petrograd_warszawa, 6).
route_points(moscow_riga, 8).

route_segment(petrograd_warszawa, petrograd_vilna).
route_segment(petrograd_warszawa, warszawa_vilna).

route_segment(moscow_riga, moscow_smolensk).
route_segment(moscow_riga, smolensk_vilna).
route_segment(moscow_riga, riga_vilna).
```

8. Занятые перегоны

```
occupies_segment(yuliya, petrograd_vilna).
occupies_segment(yuliya, warszawa_vilna).
occupies_segment(dasha, smolensk_vilna).
```

2.2 Реализация правил:

1. Свободный перегон

```
available_segment(Segment) :-
    segment(Segment),
    \+ occupies_segment(_, Segment).
```

2. Построенный перегон выбранного маршрута

```
built_route_segment(Player, Route, Segment) :-
    player_route(Player, Route),
    route_segment(Route, Segment),
    occupies_segment(Player, Segment).
```

3. Выполнение маршрута

```
all_route_segments_owned(Player, Route) :-
    \+ (route_segment(Route, Segment),
        \+ occupies_segment(Player, Segment)).

completed_route(Player, Route) :-
    player_route(Player, Route),
    all_route_segments_owned(Player, Route).
```

4. Получение очков

```
% Очки за построенный перегон
segment_points(Player, Segment, Points) :-
    occupies_segment(Player, Segment),
    segment_length(Segment, Points).

% Положительные очки за выполненный маршрут
completed_route_points(Player, Route, Points) :-
    completed_route(Player, Route),
    route_points(Route, Points).

% Отрицательные очки за невыполненный маршрут
failed_route_points(Player, Route, Points) :-
    player_route(Player, Route),
    \+ completed_route(Player, Route),
    route_points(Route, RoutePoints),
    Points is -RoutePoints.

% Сумма очков за все построенные перегоны
total_segment_points(Player, Total) :-
    findall(Points, segment_points(Player, _, Points), PointsList),
    sum_list(PointsList, Total).

% Итоговая сумма очков с учётом маршрутных карт
earned_points(Player, Total) :-
    total_segment_points(Player, SegmentPoints),
    findall(Points, completed_route_points(Player, _, Points), CompletedPoints),
    findall(Points, failed_route_points(Player, _, Points), FailedPoints),
    sum_list(CompletedPoints, CompletedTotal),
    sum_list(FailedPoints, FailedTotal),
    Total is SegmentPoints + CompletedTotal + FailedTotal.
```

5. Построение перегона

```
player_cards(yuliya, blue, 4).
player_cards(yuliya, red, 2).
player_cards(yuliya, white, 4).
player_cards(dasha, green, 4).
player_cards(dasha, orange, 2).
player_cards(dasha, yellow, 3).

can_build_segment(Player, Segment) :-
    segment(Segment),
    segment_color(Segment, Color),
    segment_length(Segment, Length),
    player_cards(Player, Color, Cards),
    Cards >= Length,
    available_segment(Segment).
```

6. Построение станций

```
station_built(yuliya, moscow).

can_build_station(Player, City) :-
    player(Player),
    city(City),
    segment_connects(Segment, City, _),
    occupies_segment(OtherPlayer, Segment),
    Player \= OtherPlayer.
```

7. Использование перегона

```
can_use_segment(Player, Segment, FromCity) :-
    occupies_segment(Player, Segment),
    segment_connects(Segment, City1, City2),
    (FromCity = City1 ; FromCity = City2).

can_use_segment(Player, Segment, FromCity) :-
    station_built(Player, FromCity),
    segment_connects(Segment, FromCity, _),
    occupies_segment(OtherPlayer, Segment),
    Player \= OtherPlayer.
```

2.3 Реализация запросов

1. Проверка существования города

```
?- city(moscow).
```

2. Показать цвет каждого игрока

```
?- player_color(Player, Color).
```

3. Проверка двух фактов одновременно (логическое И)

```
?- player(yuliya), player_color(yuliya,  
black).
```

4. Логическое ИЛИ

```
?- player_color(Player, black) ;  
   player_color(Player, purple).
```

5. Логическое НЕ

```
?- \+ available_segment(petrograd_vilna).
```

6. Проверка выполнения маршрута с использованием правила

```
?- built_route_segment(yuliya,  
    petrograd_warszawa, Segment).
```

7. Получение количества очков с использованием правила и переменной

```
?- earned_points(yuliya, Points).
```

8. Проверка возможности построить перегон

```
?- can_build_segment(Player, Segment).
```

3. Оценка и интерпретация результатов

3.1 Результаты выполнения запросов:

1. Проверка существования города

```
≡ ?- city(moscow) .
```

```
true
```

В результате выполнения запроса получен ответ `true`, что означает наличие факта `city(moscow)` в базе знаний. Следовательно, Prolog подтвердил существование города Moscow

2. Показать цвет каждого игрока

```
≡ ?- player_color(Player, Color) .
```

```
Color = black,  
Player = yuliya  
Color = purple,  
Player = dasha
```

В результате выполнения запроса Prolog последовательно возвращает значения переменных для каждого игрока. Получены следующие результаты:

```
Player = yuliya, Color = black
```

```
Player = dasha, Color = purple
```

Это показывает, что с помощью переменных можно получить информацию о цвете фишек каждого игрока

3. Проверка двух фактов одновременно (логическое И)

```
≡ ?- player(yuliya), player_color(yuliya, black) .
```

```
true
```

В результате выполнения запроса получен ответ `true`, что означает одновременное выполнение обоих условий. Игрок `yuliya` присутствует в базе знаний, а её цвет фишек указан как `black`. Запятая между условиями обозначает логическое И

4. Логическое ИЛИ

```
≡ ?- player_color(Player, black) ; player_color(Player, purple).
```

```
Player = yuliya  
Player = dasha
```

В результате выполнения запроса Prolog получил два решения: `Player = yuliya` и `Player = dasha`, что означает выполнение одного из двух условий для каждого игрока. Для Юлии выполняется условие `player_color(yuliya, black)`, а для Даши - `player_color(dasha, purple)`

5. Логическое НЕ

```
≡ ?- \+ available_segment(petrograd_vilna).
```

```
true
```

В результате выполнения запроса получен ответ `true`, что означает отрицание условия `available_segment(petrograd_vilna)`, следовательно, перегон `petrograd_vilna` занят

6. Проверка выполнения маршрута с использованием правила

```
≡ ?- built_route_segment(yuliya, petrograd_warszawa, Segment).
```

```
Segment = petrograd_vilna  
Segment = warszawa_vilna
```

В результате выполнения запроса Prolog получил два решения: `Segment = petrograd_vilna` и `Segment = warszawa_vilna`, что означает наличие двух перегонов маршрута `petrograd_warszawa`, построенных игроком `yuliya`. Результат получен с использованием правила `built_route_segment`, которое определяет перегоны выбранного маршрута, принадлежащие указанному игроку

7. Получение количества очков с использованием правила и переменной

```
≡ ?- earned_points(yuliya, Points).
```

```
Points = 13
```

В результате выполнения запроса получен результат `Points = 13`, что означает итоговое количество очков игрока `yuliya`. В расчёте учитываются 4 очка за построенный перегон `petrograd_vilna`, 3 очка

за построенный перегон `warszawa_vilna` и 6 дополнительных очков за выполнение маршрута `petrograd_warszawa`

8. Проверка возможности построить перегон

```
≡ ?- can_build_segment(Player, Segment).  
  
Player = yuliya,  
Segment = petrograd_moscow  
Player = dasha,  
Segment = moscow_smolensk  
Player = dasha,  
Segment = riga_vilna
```

В результате выполнения запроса Prolog нашел несколько подходящих вариантов: игрок `yuliya` может построить перегон `petrograd_moscow`, а игрок `dasha` может построить перегоны `moscow_smolensk` и `riga_vilna`. Это означает, что для найденных вариантов выполняются условия правила `can_build_segment`: у игрока имеется необходимое количество карт соответствующего цвета, а выбранный перегон является свободным

3.2 Анализ результатов выполнения запросов

В ходе выполнения запросов была проверена работа разработанной базы знаний. Простые запросы позволили проверить наличие отдельных фактов, например существование городов и принадлежность цвета игроку

Запросы с переменными продемонстрировали возможность получения нескольких решений из базы знаний. При наличии нескольких подходящих вариантов Prolog последовательно возвращает найденные решения. Использование логических операторов позволило сформировать составные условия с операторами «и», «или» и отрицанием

Запросы, использующие правила логического вывода, позволили получить новые сведения на основе исходных фактов. В частности, система определила перегоны, входящие в выполненный маршрут игрока, возможные для строительства перегоны и итоговое количество очков с учётом очков за построенные перегоны и бонуса за выполнение маршрутной карты

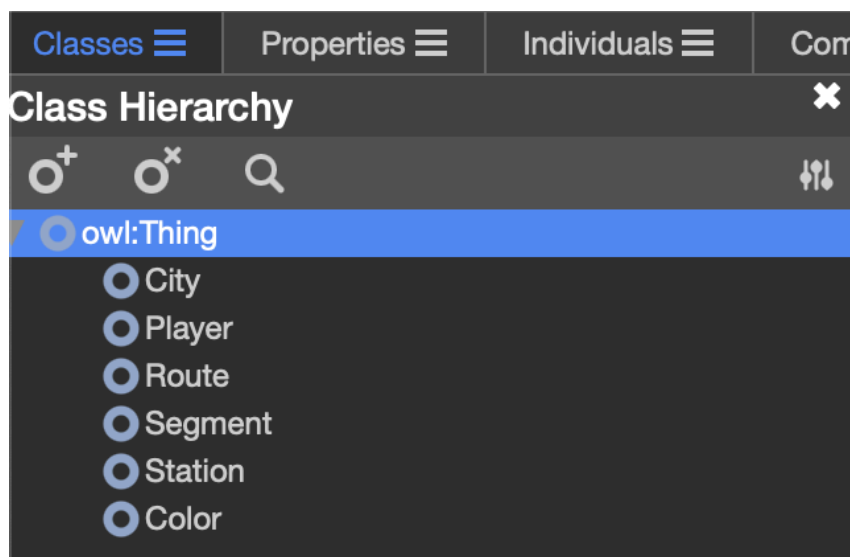
Таким образом, выполненные запросы подтвердили корректность созданных фактов, отношений и правил, а также продемонстрировали механизм логического вывода Prolog и возможность получения нескольких решений для одного запроса

4. Часть 2. Создание онтологии в Protege

4.1 Создание классов онтологии

Для перевода базы знаний из Prolog в онтологическую форму была создана онтология предметной области настольной игры Ticket to Ride в среде Protege. В качестве корневого класса использован стандартный класс `owl:Thing`, под которым были созданы основные классы предметной области

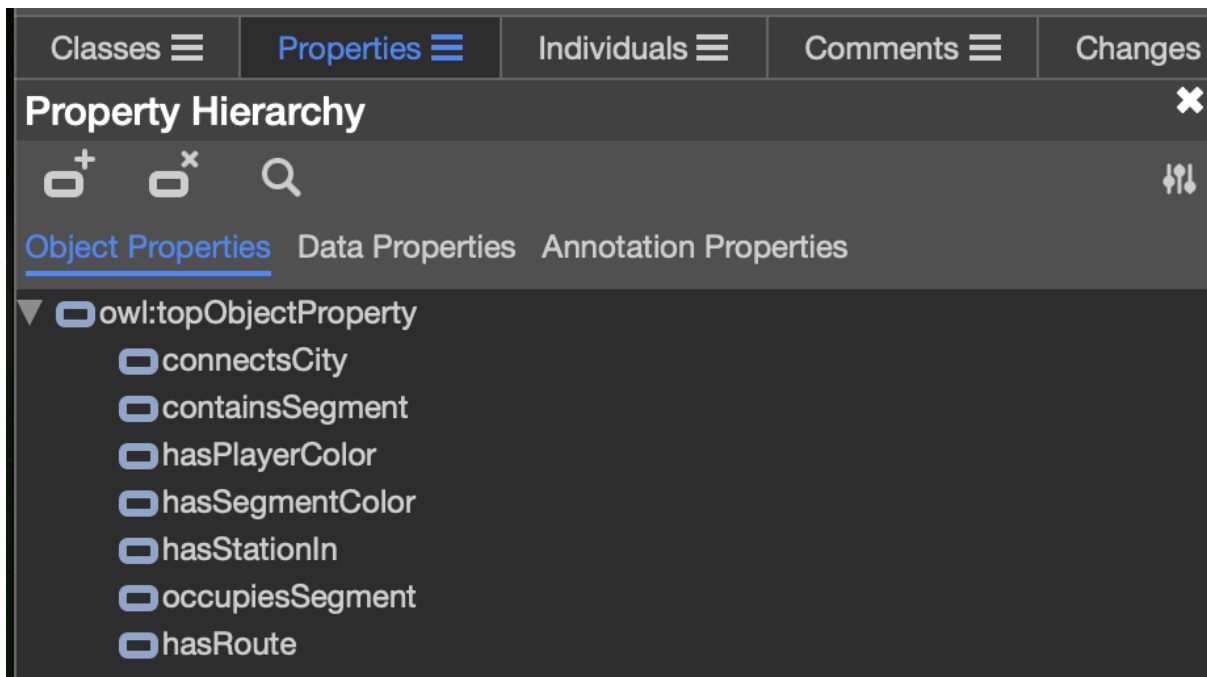
В онтологии были созданы следующие классы: `City` предназначен для представления городов игровой карты, `Player` - игроков, `Color` - цветов игровых элементов, `Segment` - отдельных перегонов между городами, `Route` - составных маршрутов, а `Station` - станций



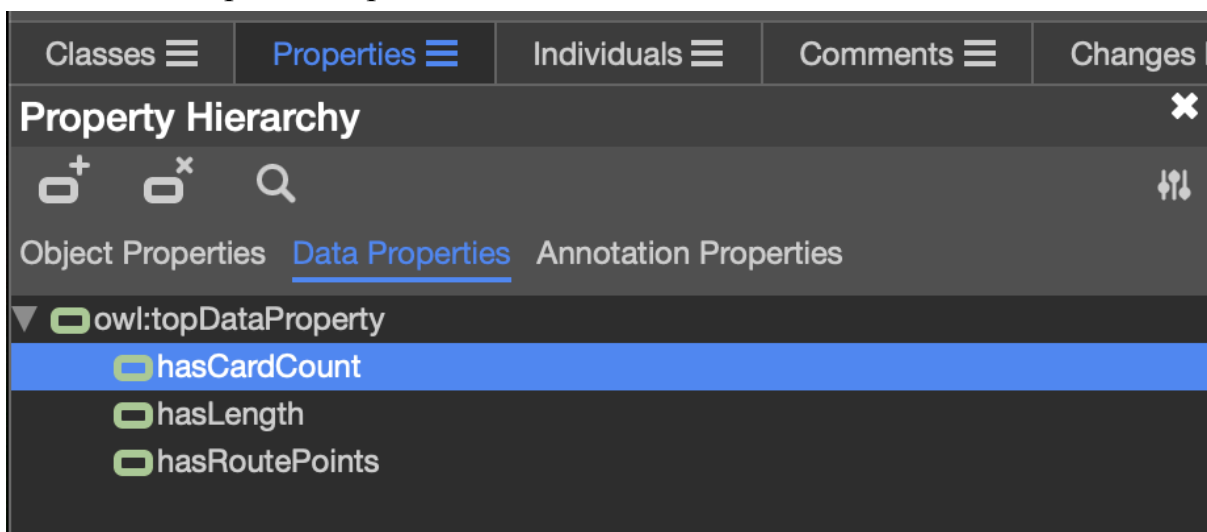
4.2 Создание свойств онтологии

После создания классов были определены свойства онтологии, соответствующие отношениям и характеристикам объектов, представленным в базе знаний Prolog

Для описания отношений между экземплярами классов были созданы следующие объектные свойства: `hasRoute`, `hasPlayerColor`, `connectsCity`, `hasSegmentColor`, `containsSegment`, `occupiesSegment` и `hasStationIn`. Данные свойства позволяют представить принадлежность маршрута игроку, цвет игрока, соединение перегона с городами, цвет перегона, состав маршрута, занятость перегона игроком и наличие станции в городе



Для представления числовых характеристик объектов были созданы следующие свойства данных: `hasLength` - длина перегона, `hasRoutePoints` - количество очков за маршрут и `hasCardCount` - количество игровых карт

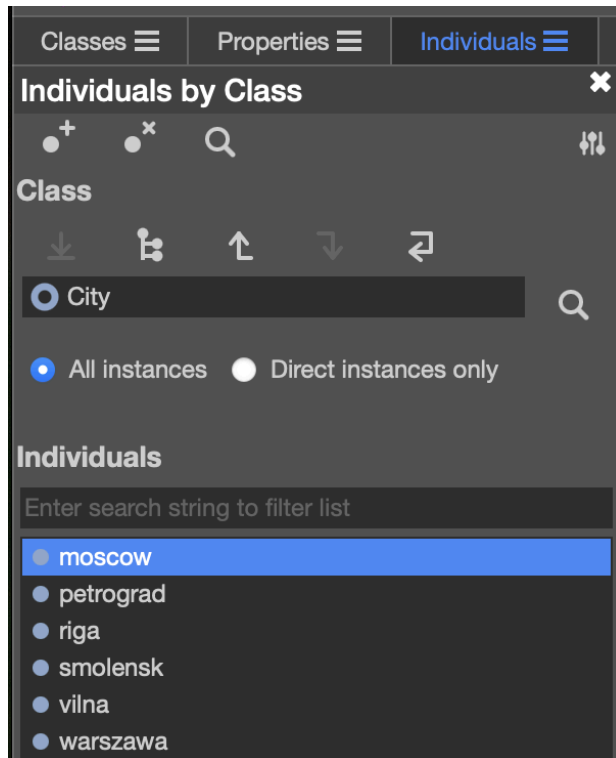


Все созданные объектные свойства были размещены на верхнем уровне иерархии объектных свойств, а свойства данных - на верхнем уровне иерархии свойств данных. Таким образом, структура свойств онтологии соответствует основным отношениям и характеристикам, использованным в базе знаний Prolog

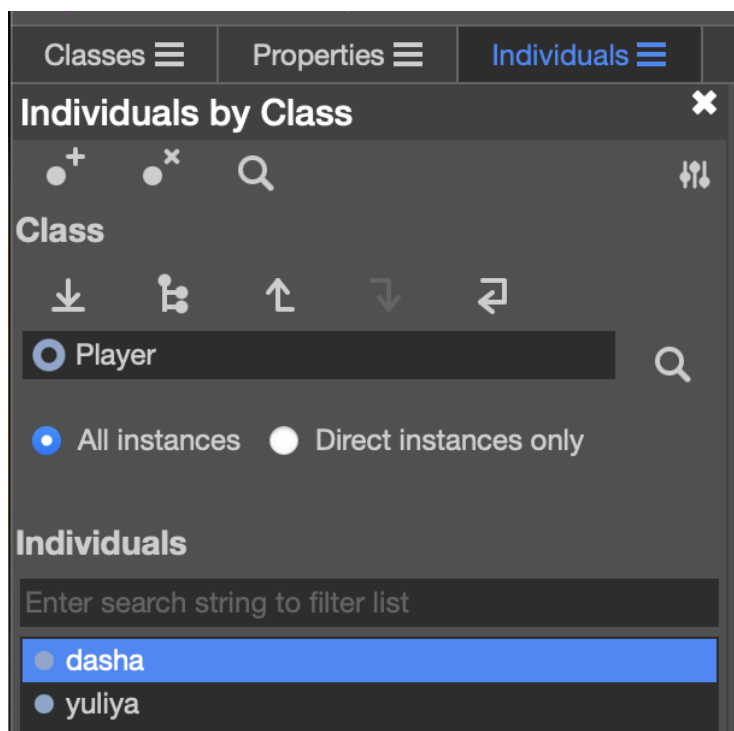
4.3 Создание экземпляров классов

После создания классов и свойств были добавлены экземпляры классов, соответствующие конкретным объектам из базы знаний Prolog

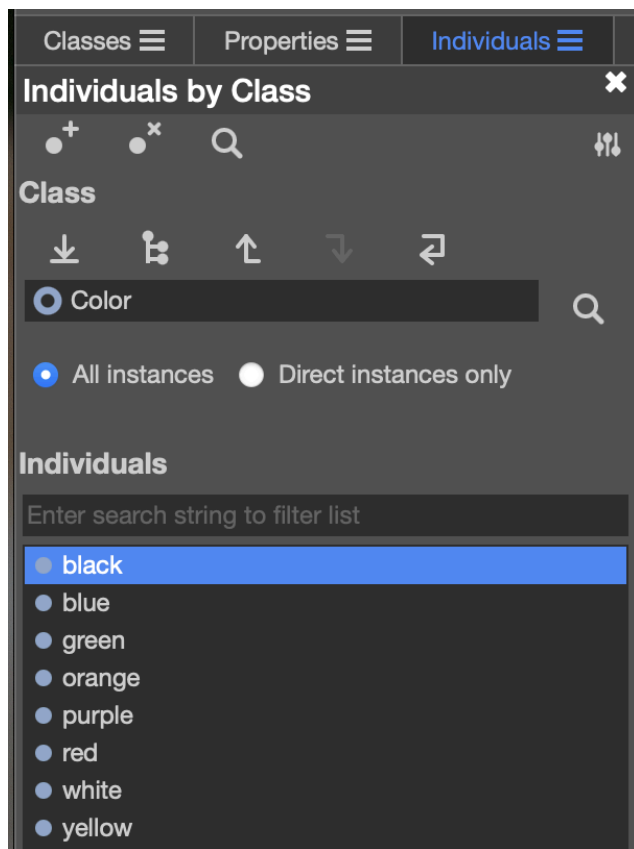
Для класса `City` были созданы экземпляры `petrograd`, `moscow`, `warszawa`, `riga`, `smolensk` и `vilna`



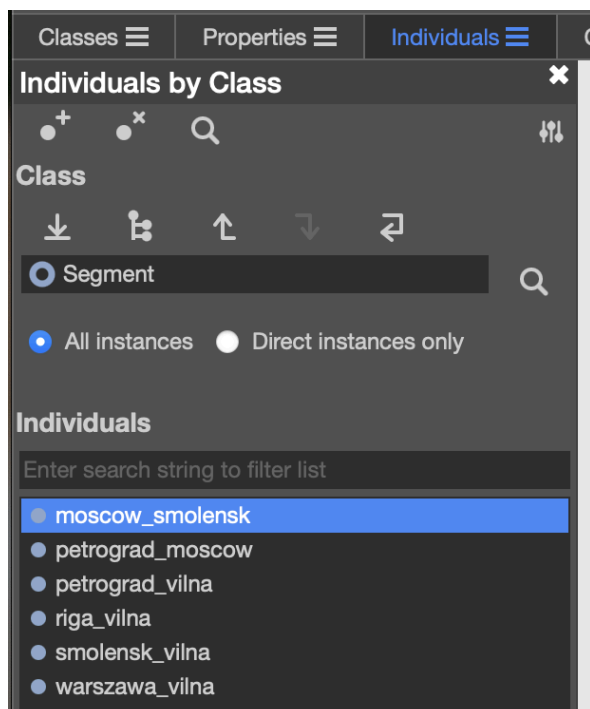
Для класса `Player` созданы экземпляры `yuliya` и `dasha`



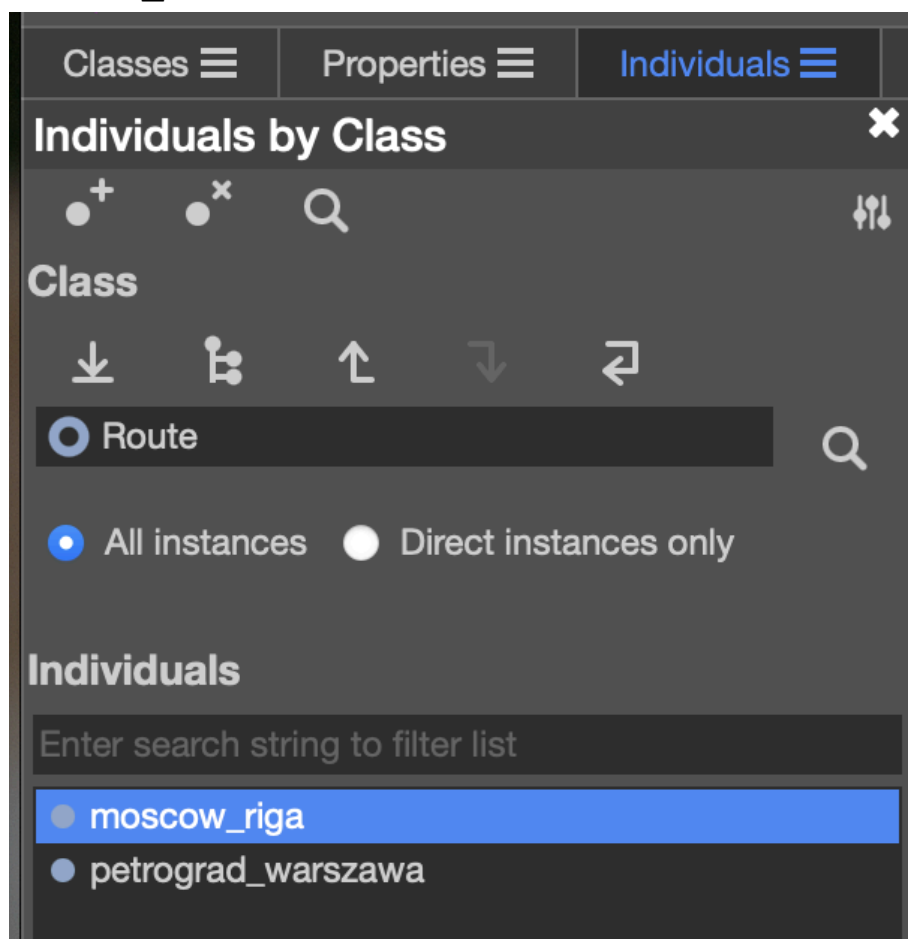
Для класса Color были созданы экземпляры red, blue, green, yellow, orange, white, black и purple



Для класса Segment созданы экземпляры petrograd_moscow, petrograd_vilna, moscow_smolensk, smolensk_vilna, warszawa_vilna и riga_vilna



Для класса `Route` созданы экземпляры `petrograd_warszawa` и `moscow_riga`

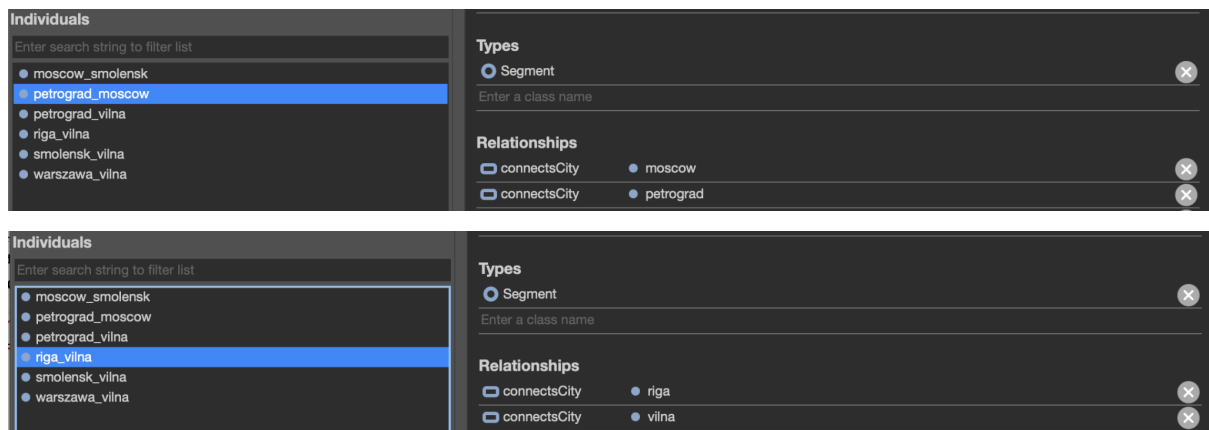


Класс `Station` также был создан в онтологии, однако отдельный экземпляр станции не создавался, поскольку в используемой модели факт наличия станции представлен отношением `hasStationIn` между игроком и городом

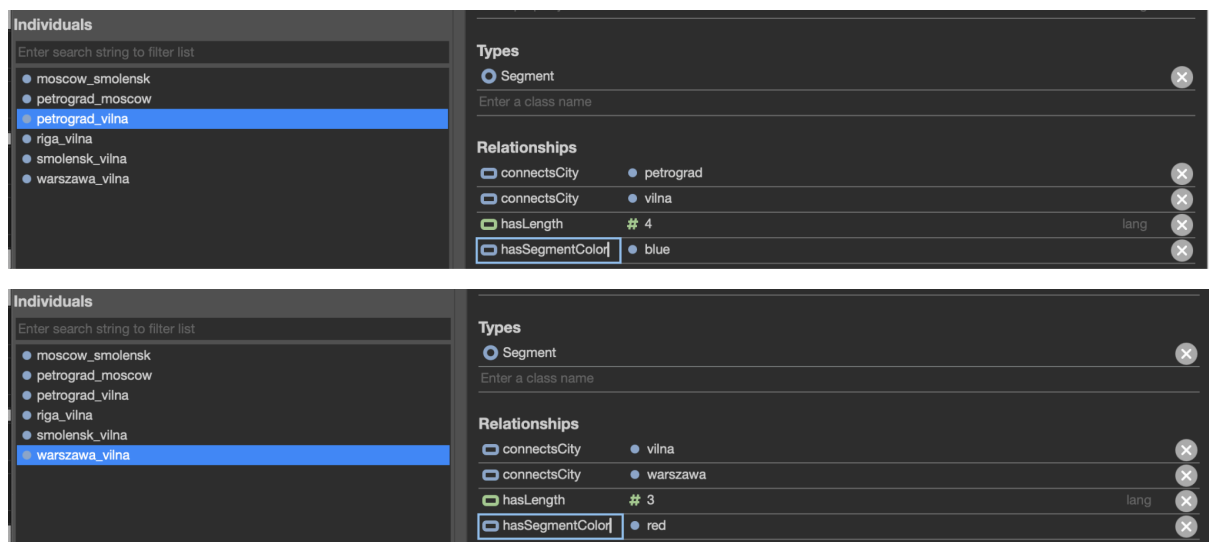
4.4 Заполнение отношений и значений

После создания экземпляров классов были заполнены отношения между объектами онтологии и заданы их числовые характеристики. Для этого использовались созданные объектные свойства и свойства данных

С помощью свойства `connectsCity` для каждого перегона были указаны соединяемые им города. Для `petrograd_moscow` заданы города `petrograd` и `moscow`, а для `riga_vilna` - города `riga` и `vilna`

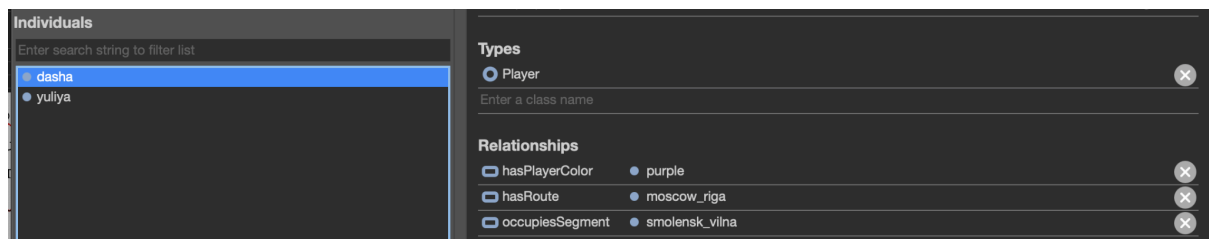
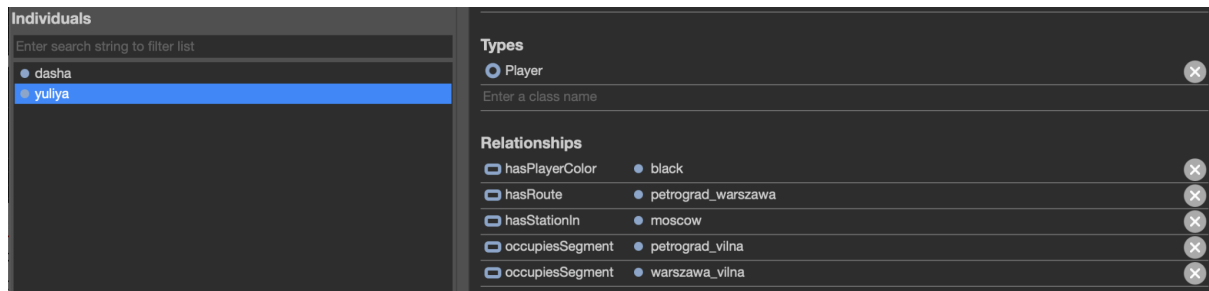


Свойство `hasSegmentColor` использовано для указания цвета каждого перегона. Перегону `petrograd_vilna` соответствует цвет `blue`, а перегону `warszawa_vilna` - цвет `red`

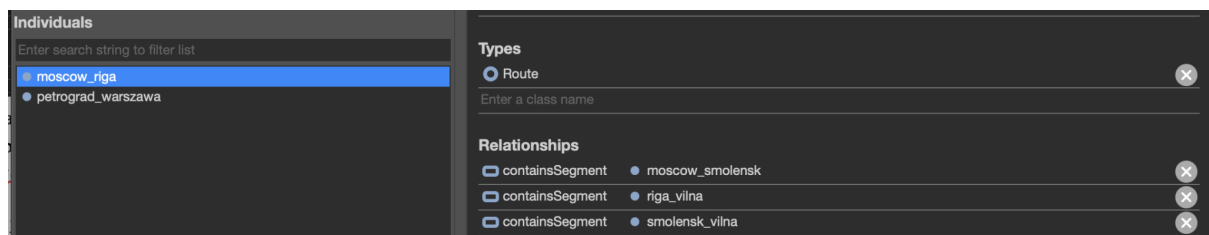
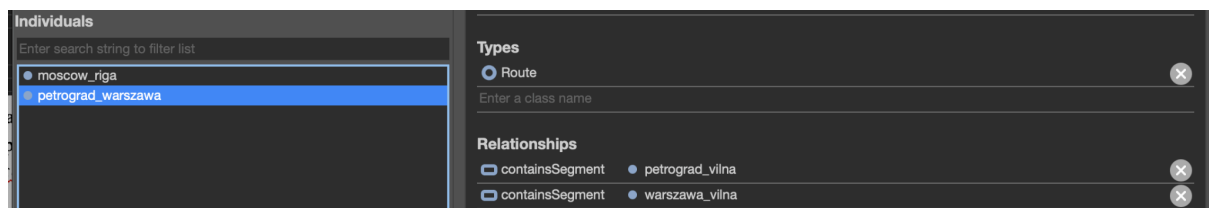


Для игроков были заполнены отношения `hasRoute`, `hasPlayerColor` и `occupiesSegment`. Игроку `yuliya` соответствует маршрут `petrograd_warszawa`, цвет `black` и два занятых перегона `petrograd_vilna` и `warszawa_vilna`. Для игрока

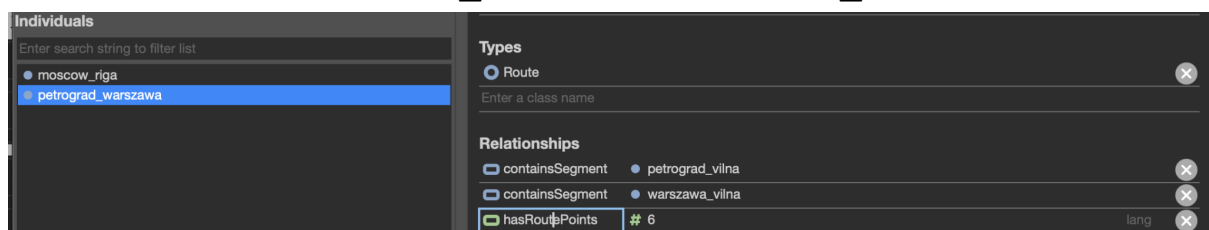
dasha задан маршрут `moscow_riga`, цвет `purple` и занятый перегон `smolensk_vilna`

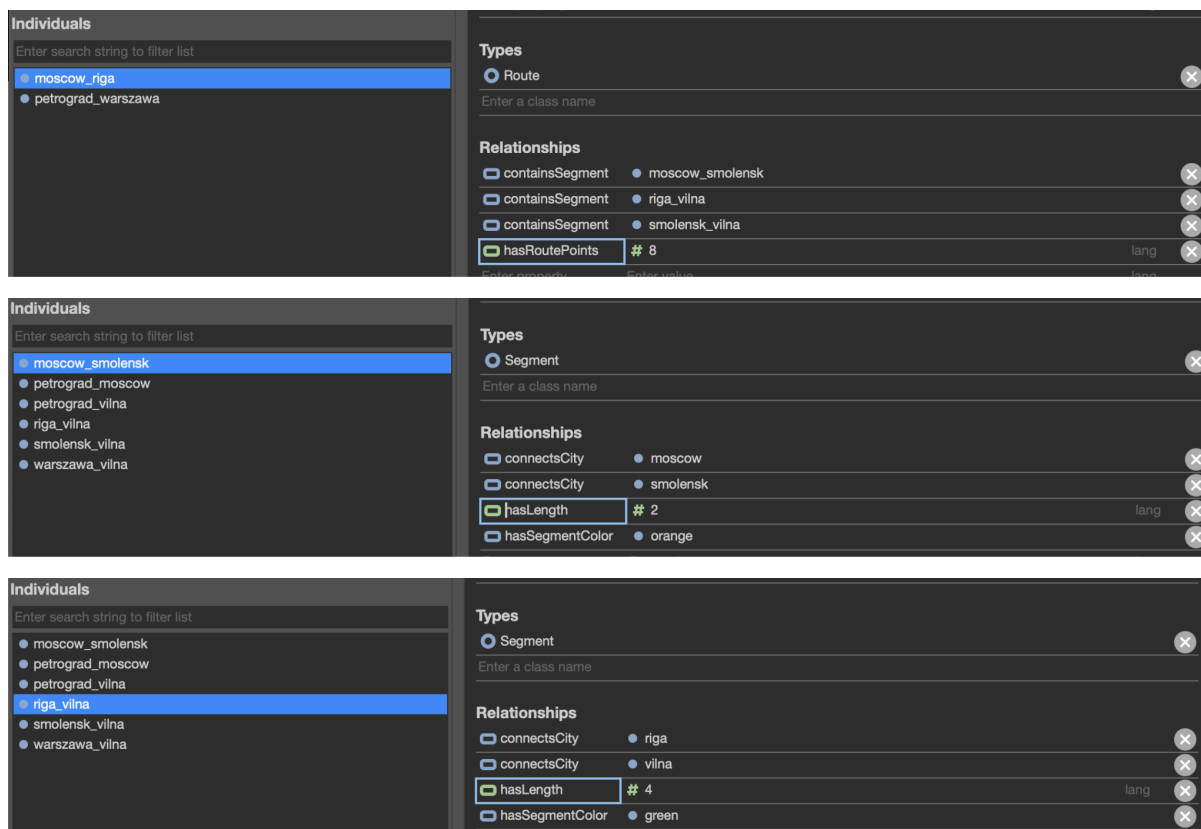


С помощью свойства `containsSegment` составные маршруты были связаны с входящими в них перегонами. Для маршрута `petrograd_warszawa` указаны перегоны `petrograd_vilna` и `warszawa_vilna`, а для маршрута `moscow_riga` - `moscow_smolensk`, `smolensk_vilna` и `riga_vilna`

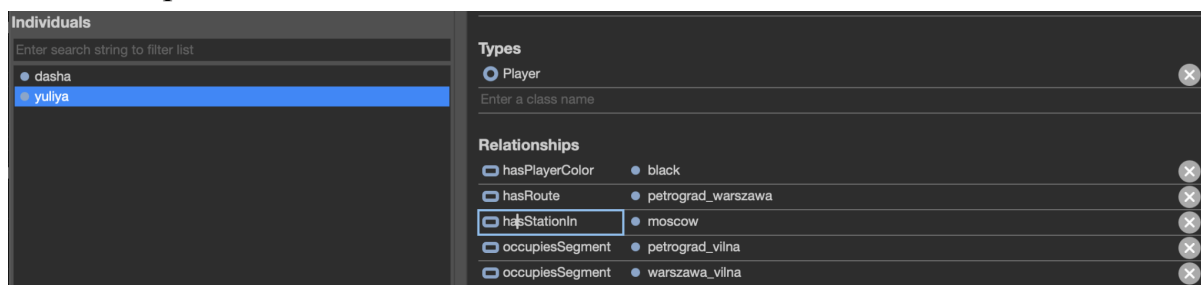


Для задания числовых характеристик использовались свойства данных `hasLength` и `hasRoutePoints`. Свойством `hasLength` были заданы длины всех перегонов, а свойством `hasRoutePoints` - количество очков для маршрутов `petrograd_warszawa` и `moscow_riga`





Для игрока *yuliya* также было задано отношение *hasStationIn* со значением *moscow*, что соответствует наличию станции игрока в данном городе

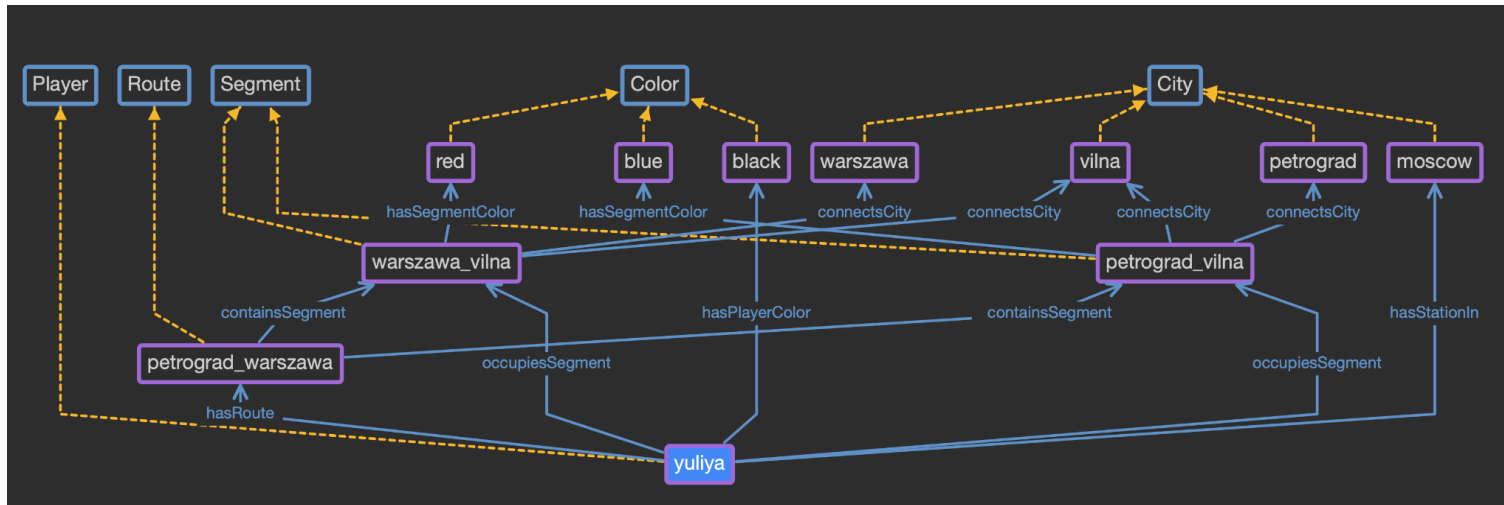


Свойство *hasCardCount* в текущей модели не заполнялось, поскольку в исходной базе знаний количество карт определяется одновременно игроком и цветом карты, а созданное свойство данных содержит только одно значение количества и не позволяет сохранить эту зависимость без введения дополнительного класса

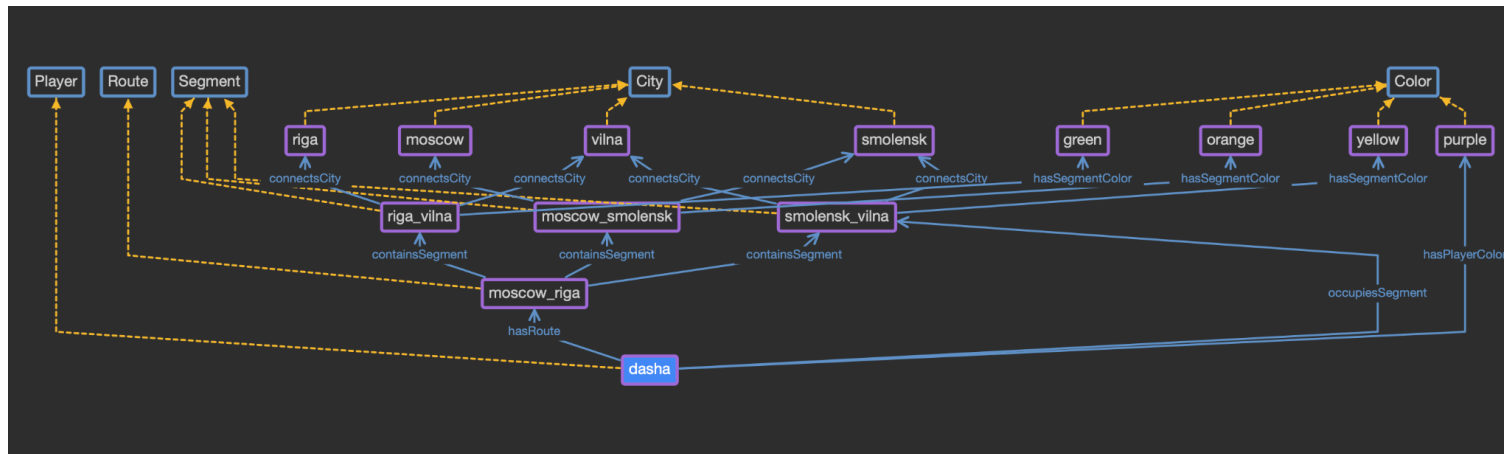
4.5 Визуализация и проверка онтологии

Для проверки созданной онтологии была использована встроенная визуализация объектов в Protege

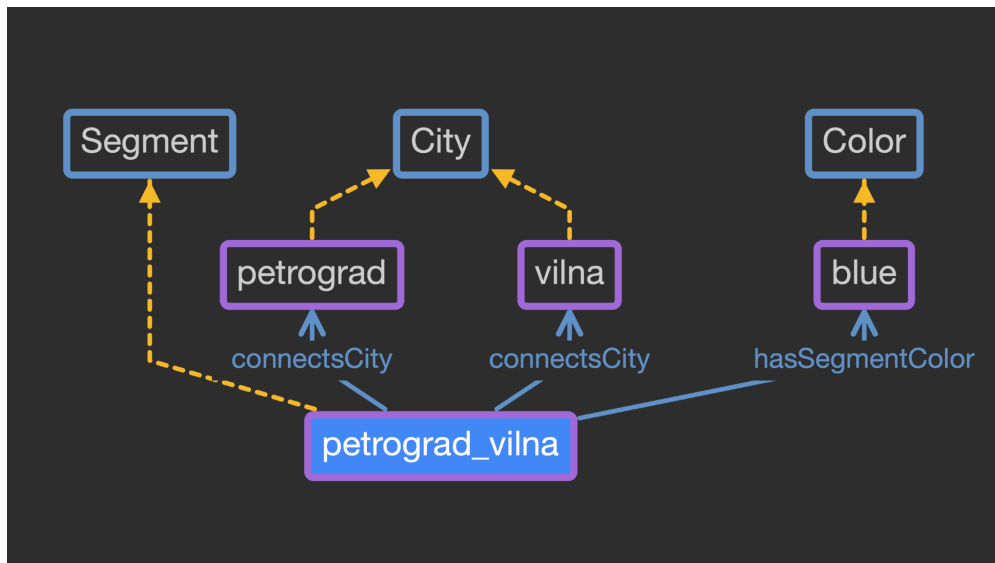
Для игрока `yuliya` отображаются связанные с ним маршрут `petrograd_warszawa`, цвет, занятые перегоны, а также город `moscow`, в котором расположена станция игрока



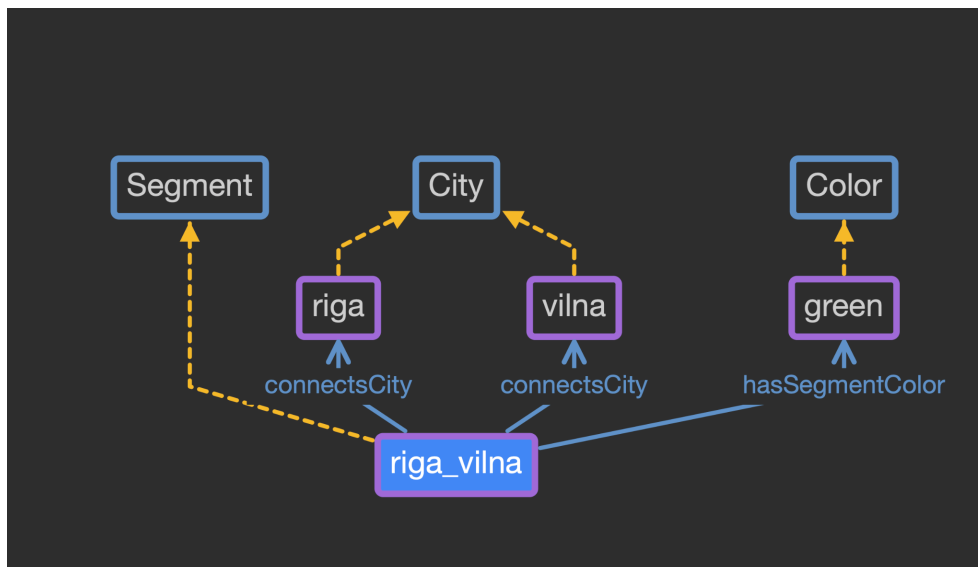
Для игрока `dasha` визуализация позволяет увидеть связанные с ним объекты, включая маршрут `moscow_riga`, цвет `purple` и занятый перегон `smolensk_vilna`



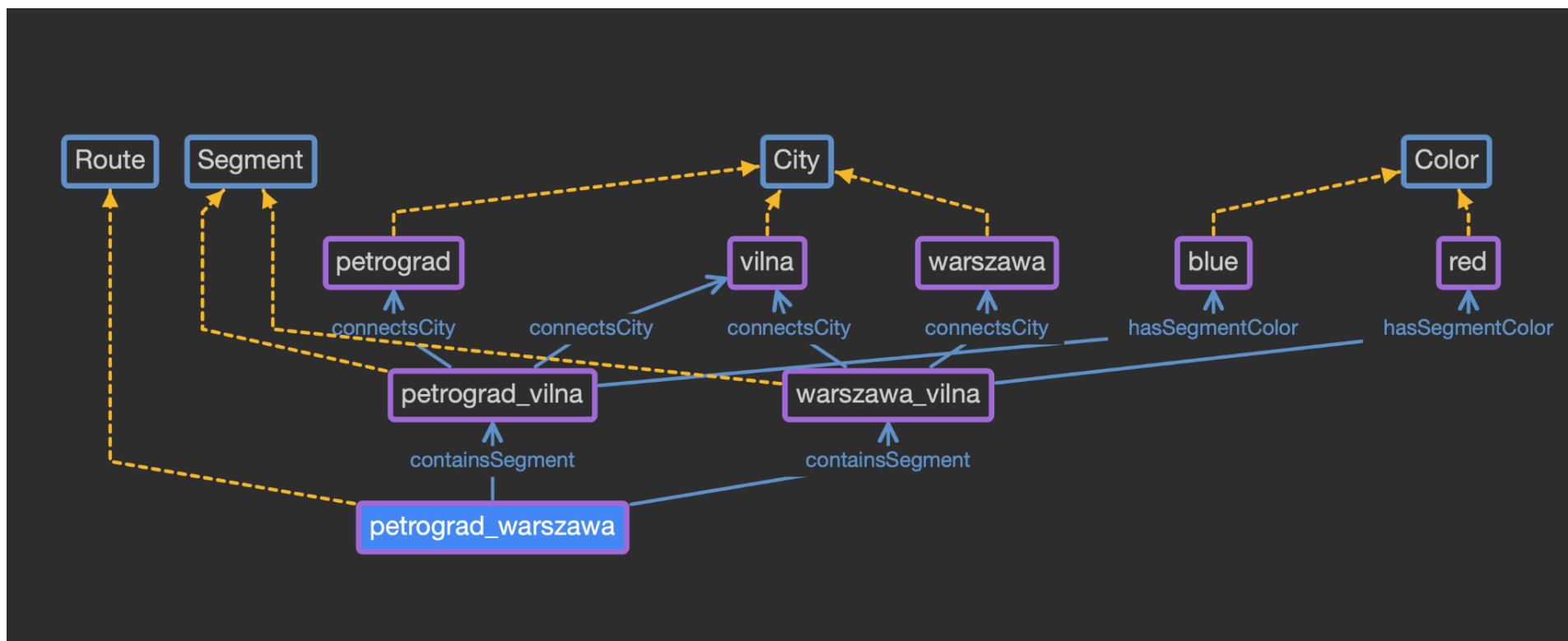
Также были проверены отдельные перегоны. Для petrograd_vilna отображаются связанные города petrograd и vilna, а также цвет перегона и его длина



Для riga_vilna отображаются города riga и vilna и соответствующие характеристики перегона



Отдельно была проверена визуализация маршрута `petrograd_warszawa`, для которого отображаются входящие в него перегоны `petrograd_vilna` и `warszawa_vilna`, а также количество маршрутных очков



Полученная визуализация подтверждает, что созданные экземпляры классов связаны между собой в соответствии с отношениями, заданными в онтологии

Заключение

В ходе лабораторной работы была разработана система представления знаний для предметной области настольной игры Ticket to Ride. Работа была выполнена в двух частях: сначала была создана база знаний на языке Prolog, а затем выполнен её перевод в онтологическую форму с использованием среды Protege

В первой части были представлены основные объекты предметной области, отношения между ними и правила логического вывода. С помощью запросов была проверена корректность базы знаний, продемонстрирована работа переменных и логических операторов, а также получение новых сведений на основе созданных правил

Во второй части была создана онтология, включающая классы, свойства и экземпляры, соответствующие объектам и отношениям исходной базы знаний. Заполнение связей и значений, а также визуализация экземпляров позволили проверить корректность созданной онтологической модели

В результате работы были изучены два подхода к представлению знаний: логическое представление в Prolog и структурированное представление в виде онтологии. Полученные результаты показали, что оба подхода позволяют формализовать знания о предметной области, при этом Prolog удобен для выполнения логического вывода и запросов, а онтология - для структурирования и визуального представления объектов и отношений

Поставленная цель лабораторной работы была достигнута, а разработанная система может быть расширена за счёт добавления новых объектов, свойств, правил и игровых состояний